



MANUAL TEST GUIDE · MODULE 07 OF 11

Consolidation Functional-Flow Test Walkthroughs

Scenario-driven QA guide for **logistics.consolidation** — the orchestration layer that groups many house shipments under one master **console** and enforces capacity, cut-off, and mode-aware manifest discipline. Grounded on the SGSIN LCL groupage lane out of Mumbai.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds. **Phase-1 honesty:** house-linking is Phase 2, so capacity `used` is 0 and a fresh master yields 0 manifest lines until houses are linked — noted where it matters.

01 What Consolidation does & how to read this guide

Consolidation is “many shipments sharing one equipment usage.” The module owns very little cargo data — it sits on top of Shipments (master + houses) and Equipment-Ops (the shared `equipment.usage`). It owns the **console** (the orchestration header + status lifecycle), the **policy / type** reference masters, and three computed projections: **eligibility**, **capacity snapshot**, and the mode-aware **manifest**.

9

COMMANDS TO EXERCISE

3

TEST SCENARIOS

5

CONSOLE LIFECYCLE STATES

8

CONSOLIDATION TYPES

The state machines you will observe

The **console** carries the governed lifecycle; two derived enums track cut-off discipline and manifest issue. Watch all three during Scenarios B and C.

```
console.status : draft → planning → confirmed → frozen | cancelled (from draft / planning / confirmed only)
cutoff_state   : open → soft_closed → hard_closed (soft = etd - consol.cutoff_hours_before_departure)
manifest.status : draft → generated → issued | superseded · eligibility: eligible / eligible_with_warning / ineligible
```

The nine commands, grouped by scenario

CONFIG & PROJECTIONS (A / C)

- `eligibility.recompute` — per-shipment verdict
- `capacity.snapshot.recompute` — capacity remaining
- `manifest.generate` · `manifest.issue`
- Pool = a saved search over `consolidation.eligibility` via `consolidation.pool.filter`

CONSOLE LIFECYCLE (B)

- `console.set_planning` — draft → planning
- `console.confirm` — completeness gate + auto-master
- `console.evaluate_cutoff` — recompute cut-off state
- `console.freeze` · `console.cancel`

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Config & eligibility	Policy / type masters, scope-priority resolve, per-shipment eligibility verdict, saved pool filter	<code>eligibility.recompute</code>	§02
B — Console lifecycle	<code>set_planning</code> → <code>confirm</code> → <code>freeze</code> , auto-master, capacity snapshot, cut-off discipline + guards	<code>set_planning</code> , <code>confirm</code> , <code>freeze</code>	§03 · §04
C — Manifest	Mode-aware doc-type, house-line aggregation, supersede on regenerate, issue gate	<code>manifest.generate</code> , <code>manifest.issue</code>	§05

Demo fixtures you start from (loaded by `--with-demo=all`)

Org **Acme Forwarding — Mumbai** (`base.default_organization`). Standard seed: 3 tenant-wide policies (`GROUP-BY-POD` / `CUSTOMER` / `CARRIER`) + 8 `consolidation.type` defaults. Branch demo: `DEMO-SGSIN-LCL` (destination-grouping, 36 h cut-off, 10% buffer) and `DEMO-AIR-CONSOLE` (customer-grouping, 12 h cut-off), plus a **draft** console `consolidation.demo_console_sgsin_lcl` (type `lcl_groupage` , mode `sea`) — Scenario B drives it forward.

02 Config masters, scope resolution & eligibility

Goal: confirm the two reference masters seed correctly, that a branch-scoped policy beats the tenant-wide default (BR-PM-01), and that the per-shipment eligibility projection computes a real verdict. **Role:** Consolidation coordinator.

#	Action	Expected result	✓ Checkpoint	Trace
1	Open the Consolidation Policies list.	5 rows: 3 tenant-wide (<code>GROUP-BY-POD/CUSTOMER/CARRIER</code>) + 2 Mumbai-branch (<code>DEMO-SGSIN-LCL</code> , <code>DEMO-AIR-CONSOLE</code>). Order = <code>priority desc</code> .	Branch rows carry <code>organization_id</code> ; defaults are null.	CON-04
2	Open the Consolidation Types list (8 rows).	<code>lcl_groupage</code> → sea, <code>air_console</code> → air; <code>export_console</code> / <code>import_deconsole</code> / <code>multimodal_console</code> have no mode (mode-agnostic). Each carries a default equipment family + manifest doc-type.	Mode-bound vs mode-agnostic split is correct.	BR-CT-02
3	Resolve a policy for the demo lane: <code>group_by=destination</code> , <code>organization_id=Mumbai</code> , <code>transport_mode_id=sea</code> .	<code>DEMO-SGSIN-LCL</code> wins over the tenant-wide <code>GROUP-BY-POD</code> — highest priority, then most-specific scope.	Org-scoped row beats the wildcard default.	BR-PM-01
4	Note the demo policy's own <code>capacity_buffer_pct=10</code> / <code>cutoff_hours_before_departure=36</code> .	These override the org <code>ir.config</code> defaults (5% / 24 h) for consoles created under the policy.	Policy override values read back as seeded.	BR-PM-02
5	Run <code>consolidation.eligibility.recompute</code> with the seeded Globex export shipment's <code>shipment_id</code> .	An eligibility row is upserted: <code>is_eligible</code> + <code>eligibility_state</code> computed from real fields (status, POL/POD, validation_state); pool dims (weight / volume / hazardous / reefer) rolled up from cargo lines.	One row per shipment; verdict populated.	BR-EL-01/02
6	Recompute against a cancelled shipment and read <code>blocking_reasons</code> .	<code>eligibility_state=ineligible</code> ; the reason list names <code>status_ok</code> → “shipment status does not permit consolidation (cancelled / terminal)”. Every failed criterion is listed, not just the first.	All-reasons list, not first-fail.	BR-EL-04
7	Build a pool filter — save a <code>consolidation.pool.filter</code> over eligibility (e.g. <code>is_eligible=true</code> , POD = SGSIN). Toggle <code>is_shared</code> .	Filter saved; private to owner until <code>is_shared=true</code> , then visible branch-wide. The pool is a search view over eligibility — no separate pool entity.	Shared toggle changes visibility scope.	BR-PS-03
8	Negative: call <code>eligibility.recompute</code> with no <code>shipment_id</code> .	Rejected: “recompute requires a shipment_id”.	Blocked.	guard

✓ Scenario A pass criteria

Both masters seeded, the org-scoped policy wins the resolve, the eligibility verdict is computed from real shipment data with a full reasons list, and a shareable pool filter persists.

⚠ Phase-1 safe-pass criteria

Four eligibility criteria (`cargo_ready_ok` , `cutoff_ok` , `compatibility_ok` , `hold_status_ok`) **safe-pass** today — the holds / CON-08 compatibility data lands in Phase 2. Don't read a green verdict as “all seven criteria evaluated”; only status / route / doc-readiness carry real signal now.

03 Console lifecycle — planning → confirm → freeze

Goal: drive the demo console through its full governed lifecycle, watch the auto-master + capacity snapshot appear on confirm, and freeze it under clean cut-offs. **Start:** `consolidation.demo_console_sgsl_lcl` in **draft** (type `lcl_groupage`, mode `sea`).

#	Action	Expected result	✓ Checkpoint	Trace
1	Read the console's <code>console_number</code> .	Auto-stamped from <code>ir.sequence('CONS')</code> → <code>CONS-2026-NNNNNN</code> ; immutable.	Number present, not blank.	BR-CC-03
2	Fill the confirm-required fields the demo leaves blank: <code>pol_id</code> , <code>pod_id</code> , <code>carrier_id</code> , <code>equipment_type_id</code> (mode is already <code>sea</code>).	Fields accept; console still draft .	All five required fields now set.	BR-CC-05
3	Run <code>consolidation.console.set_planning</code> .	Status draft → planning ; <code>planning_at</code> stamped.	Status = planning .	BR-CC-01
4	Run <code>consolidation.console.confirm</code> .	Completeness gate passes → status confirmed , <code>confirmed_at</code> stamped. With <code>consol.auto_create_master_on_confirm=true</code> a master <code>logistics.shipment</code> (<code>hierarchy_type=master</code>) is created and linked; a <code>consolidation.console.confirmed</code> event fires.	<code>master_shipment_id</code> now set.	BR-CC-04/05
5	Open the Capacity Snapshot for the master (auto-built by the confirm event).	A <code>consolidation.capacity.snapshot</code> row exists — totals denormalized from the equipment type, buffer resolved from the policy (10%), <code>used=0</code> , <code>remaining = total × (1 - buffer)</code> .	Snapshot present; buffer = policy's 10%.	BR-CAP-01/03
6	Run <code>consolidation.console.evaluate_cutoff</code> (set <code>etd + cargo_cut_off</code> first).	<code>cutoff_state</code> + <code>soft_cutoff_at</code> recomputed: <code>soft_cutoff_at = etd - 24 h</code> ; state open before soft, soft_closed between soft and cargo cut-off, hard_closed after.	State matches the now-vs-cut-off window.	BR-CO-01/02/03
7	With all three carrier cut-offs (<code>si_cut_off</code> / <code>vgm_cut_off</code> / <code>customs_cut_off</code>) in the future, run <code>consolidation.console.freeze</code> .	Status confirmed → frozen , <code>frozen_at</code> stamped; a <code>consolidation.console.frozen</code> event fires.	Status = frozen .	BR-CO-05

✓ Scenario B pass criteria

The console walked `draft` → `planning` → `confirmed` → `frozen`; the master shipment and capacity snapshot were created automatically on confirm; cut-off state recomputed correctly. Negatives that must refuse are in §04.

⚠ Capacity `used` is 0 by design in Phase 1

House-linking (`consolidation.house.link`) is Phase 2, so no houses consume the master yet — `used_*` reads 0 and `remaining` equals the full buffered total. The over-capacity arithmetic (BR-CAP-02) is implemented and unit-tested; it becomes live the moment real `used` numbers arrive.

04 Lifecycle & cut-off guards — each must refuse

Half of testing an orchestrator is confirming it **refuses** out-of-order transitions and past-cut-off freezes. Each row trips a real `ValueError` — the quoted text is the exact message from the command handler. Use a **fresh** console (or reset the demo one) so each guard is reached cleanly.

#	Action	Expected refusal (exact message)	✓	Trace
1	<code>confirm</code> a console still in draft (skip <code>set_planning</code>).	"cannot confirm a console in status 'draft' (allowed: ['planning']) — BR-CC-01".	Blocked	BR-CC-01
2	Move to planning, then <code>confirm</code> before filling POL/POD/carrier/equipment.	"cannot confirm — these must be set first: pol_id, pod_id, carrier_id, equipment_type_id (BR-CC-05)".	Blocked	BR-CC-05
3	On a confirmed console, back-date <code>si_cut_off</code> (or <code>vgm/customs</code>) to the past, then <code>freeze</code> without an override.	"cannot freeze — these carrier cut-offs are already in the past: si_cut_off (BR-CO-05; pass cutoff_override to proceed)".	Blocked	BR-CO-05
4	Re-run step 3's <code>freeze</code> passing <code>cutoff_override=true</code> in the payload.	Override accepted → status frozen ; the past cut-off is bypassed by explicit override.	Override path works	BR-CO-05
5	<code>freeze</code> a console that is still in planning (not yet confirmed).	"cannot freeze a console in status 'planning' (allowed: ['confirmed']) — BR-CC-01".	Blocked	BR-CC-01
6	<code>cancel</code> a frozen console.	"cannot cancel a console in status 'frozen' (allowed: ['confirmed', 'draft', 'planning']) — BR-CC-01". A frozen console is terminal for cancellation.	Blocked	BR-CC-01
7	Edit <code>console_number</code> on any existing console.	"console_number is immutable (BR-CC-03)".	Blocked	BR-CC-03
8	Create a console whose <code>consolidation_type</code> is mode-bound (e.g. <code>lcl_groupage</code> = sea) but set <code>transport_mode_id</code> to air.	Rejected: "consolidation_type 'lcl_groupage' is bound to a different transport mode than the console's (BR-CC-02)".	Blocked	BR-CC-02
9	Positive close: <code>cancel</code> a draft / planning / confirmed console with a <code>reason</code> .	Status cancelled , <code>cancelled_at</code> + reason stamped; a <code>consolidation.console.cancelled</code> event fires.	Cancel lands terminal.	BR-CC-01

Cut-off state engine (BR-CO-01..03)

`soft_cutoff_at` = `etd` - `consol.cutoff_hours_before_departure` (org default 24 h; a policy may override). State walks `open` → `soft_closed` → `hard_closed`, with `cargo_cut_off` as the hard boundary. Missing `etd` / `cargo_cut_off` degrade toward `open` — nothing to enforce yet.

05 Mode-aware manifest — generate, aggregate & issue

Goal: generate one manifest that rolls up every house linked to the console's master, prove the doc-type is picked by mode (never operator-typed), and gate the issue on the console being confirmed / frozen. **Start:** a **confirmed** or **frozen** console (from Scenario B) with a linked master.

#	Action	Expected result	✓ Checkpoint	Trace
1	Run <code>consolidation.manifest.generate</code> with the <code>console_id</code> .	A <code>consolidation.manifest</code> is created in generated , numbered from <code>ir.sequence('CNMF')</code> → <code>CNMF-2026-NNNNNN</code> ; <code>generated_at</code> stamped; a <code>consolidation.manifest.generated</code> event fires.	Manifest = generated, numbered.	CON-18
2	Read <code>document_type_id</code> on the manifest.	Auto-selected from the master's <code>transport_mode_id</code> (sea → sea-manifest doc-type); never operator-picked.	Doc-type matches the mode.	BR-MF-01
3	Inspect the manifest lines and totals.	One <code>consolidation.manifest.line</code> per active house on the master (<code>house_ref</code> , packages, gross, CBM, chargeable weight); header totals = the sum across lines.	Lines aggregate the houses; totals reconcile.	BR-MF-02
4	Note the chargeable weight per line (sea LCL).	Sea: <code>max(gross, cbm × 1000)</code> . (Air would use <code>cbm × 1e6 / 6000</code> ; road / rail / FCL co-load use actual gross.) Divisors come from <code>consol.sea_volumetric_factor</code> / <code>consol.air_volumetric_divisor</code> .	Chargeable weight follows the mode rule.	BR-MF-04
5	Re-run <code>manifest.generate</code> for the same master.	The prior generated manifest flips to superseded ; a fresh generated one replaces it — only one live manifest per master.	Old manifest = superseded.	BR-MF-03
6	Run <code>consolidation.manifest.issue</code> on the generated manifest.	Status generated → issued , <code>issued_at</code> stamped — allowed because the owning console is confirmed / frozen.	Manifest = issued.	BR-MF-05
7	Negative: <code>issue</code> a manifest whose console is still in draft / planning .	Rejected: "manifest cannot be issued until the console is confirmed or frozen (BR-MF-05)".	Blocked	BR-MF-05
8	Negative: <code>issue</code> a manifest that is not in generated (e.g. already issued or superseded).	Rejected: "only a generated manifest can be issued".	Blocked	guard

⚠ Phase-1 honesty — 0 lines on an auto-created master

The console's auto-master starts with **no houses** (house-linking is Phase 2), so a first `generate` yields `house_count=0` and no lines — the header, mode-aware doc-type, supersede lifecycle, and issue gate are all still exercised. To see real line aggregation now, point one or more existing `logistics.shipment.house` rows at the master (`master_shipment_id`) before generating; the engine reads active houses + their cargo lines directly.

✓ Full-module pass criteria

All three scenarios green: config masters + eligibility + pool filter (A); console draft → frozen with auto-master, capacity snapshot, cut-off discipline and every lifecycle guard refusing (B); mode-aware manifest with house aggregation, supersede-on-regenerate and a gated issue (C).

Traceability: `CON-*` IDs map to the Consolidation Management BRD scope areas (1 Eligibility · 2 Pool · 3 Console · 4 Type · 18 Reports/Manifest); `BR-CC` / `BR-CO` / `BR-CAP` / `BR-MF` / `BR-EL` / `BR-PM` / `BR-CT` / `BR-PS` are the module's feature-level business rules. House-linking, 3D load planning, and AI recommendation (BRD scope 5, 9–13) are Phase 2 and out of this guide.